

Slot storage

0xSlots protocol v1 · Slot.sol behind a BeaconProxy · v1 branch, 25 Sept 2026
Each group below, plus OZ Initializable and ReentrancyGuard, has its own ERC-7201 location.

LOCATION · STRUCT	FIELD	TYPE	HOLDS	WRITTEN BY
slots.governance Governance How the slot is governed. Fixed at creation except manager.	currency	IERC20	Payment token. Zero address = native ETH.	initialize
	manager	address	Proposes terms, accepts module updates, hands over. Set exactly when something is mutable.	initialize · setManager
	mutableTax · mutableRecipient · mutableModule	bool ×3	Which terms may ever change: tax rate and runway · recipient (and raising or redirecting a module fee) · module and its scopes.	initialize
slots.terms.tax TaxTerms · live	recipient	address	Receives collected tax, minus the module fee.	initialize · terms land
	rateBps	uint16	Tax per 30 days, in basis points of the price. 1–10,000.	initialize · terms land
	minRunwaySeconds	uint32	A buyer’s deposit must cover this long at their price. ≤ 365 days.	initialize · terms land
slots.module InstalledModule One record. Installing copies one in; removing deletes it. Same shape in Pending.nextModule.	module	address	The installed module. Zero = none.	initialize · module lands
	scopes	uint16	Copied from the module. beforeBuy 1 · beforeSelfAssess 2 · afterBuy 4 · afterRelease 8 · afterLiquidate 16 · afterSettle 32 · afterCallbacksMustSucceed 64 · onInstall 128 · onUninstall 256.	initialize · module or scopes land
	fee	ModuleFee	Copied from the module: bps (0–10,000 of collected tax) and recipient. Payouts use it.	initialize · acceptFee · module lands
	settings	bytes	abi.encode of the fields the module’s definition lists. Handed to every callback.	initialize · module lands
slots.pending Pending Stored exactly as pending() returns it. Lands at the next buy or applyTerms, 1 hour after proposal.	taxTerms	TaxTerms	Proposed tax values. Only the fields named in mask count.	proposeTerms · cancel · land
	nextModule	InstalledModule	A proposed module with the scopes and fee reviewed for it, or only newly accepted scopes. Checked again when it lands.	proposeTerms · acceptScopes · cancel · land
	mask	uint16	TAX_RATE 1 · RECIPIENT 2 · MIN_RUNWAY 4 · MODULE 8 · SCOPES 16 (never with MODULE).	propose · acceptScopes · cancel · land
	proposedAt	uint64	Start of the tax terms' delay. Proposing tax terms restarts it.	proposeTerms
	moduleProposedAt	uint64	Start of the module's delay, on its own clock.	proposeTerms · acceptScopes
slots.occupancy Occupancy Who holds the slot, at what price, funded by how much.	occupant	address	Current holder. Zero = vacant.	buy · release · liquidate
	occupiedSince	uint64	When the current tenure began.	buy
	tenureId	uint64	Counts tenures. Scopes operator approvals to one tenure.	buy
	lastSettled	uint64	Tax is accounted up to this timestamp.	every settle
	taxCarry	uint64	Remainder below one unit of currency, carried to the next settle.	every settle · buy · vacate
	price	uint256	Self-assessed price. ≤ 2 ¹²⁸ –1.	buy · selfAssess
	deposit	uint256	Escrow that tax is drawn from.	buy · topUp · withdraw · settle
slots.ledger Ledger Tax taken, and what could not be paid either way.	collectedTax	uint256	Tax taken from deposits, not yet paid out.	settle · collect · any payout
	claimableOf	address → uint256	Payouts that could not be pushed. Taken with claim.	failed payout · claim
	debtOf	address → uint256	Tax the occupant owes beyond an emptied deposit. Repaid first by topUp or out of a buyout; ends with the tenure.	settle · topUp · buy · exit
	operatorOf	tenure → addr → bool	Who may reprice for the occupant, for one tenure only.	setOperator

Terms land at the next buy or applyTerms, 1 hour after proposal.

Settle runs first in almost every call: tax moves from deposit to collectedTax; a shortfall becomes debt until the seat changes hands.

Upgrades append fields only.

Slot methods · writes

Grouped by who may call, then what the slot calls on its module.

CALLER	METHOD	DOES
Occupant The current holder. selfAssess also takes an operator.	selfAssess (newPrice)	Restate the price; tax follows from now on. The module may veto.
	withdraw (amount)	Take deposit back, keeping what the current terms require.
	release ()	Leave. The rest of the deposit is refunded.
	setOperator (operator, allowed)	Let someone reprice on your behalf, for this tenure only.
	applyTerms ()	Land ripe terms without waiting for a buy. Anyone's call once the slot is vacant. Waits while the occupant owes debt.
Anyone	buy (account, selfAssessedPrice, depositAmount, maxPayment) PAYABLE	Pay the sitting price (nothing if vacant) plus a deposit, and seat account. Lands ripe terms; the module may veto.
	topUp (amount) PAYABLE	Add to the occupant's deposit. Repays their debt first.
	liquidate ()	Evict an occupant whose deposit is empty. No bounty; the reward is a vacant slot.
	collect ()	Pay out collected tax: the module fee, then the recipient.
	claim (account)	Send account a payout that could not be pushed to them.
	multicall (bytes[] data)	Several calls in one transaction, e.g. liquidate then buy. Not payable, so ERC-20 slots only for that.
Manager Only on a slot created with something mutable.	proposeTerms (taxTerms, moduleTerms, mask)	Queue changes; they wait 1 hour. Checks the module's settings and keeps the scopes and fee it reviewed.
	cancelTerms (mask)	Drop the named queued changes, leaving the rest.
	acceptFee (ModuleFee expected)	Take the fee the module declares now. Applies at once; a rise or a new fee recipient needs mutableRecipient. Waits while the occupant owes debt.
	acceptScopes (uint16 expected)	Take the scopes the module declares now. Queued for the next buy; needs mutableModule.
	setManager (next)	Hand the slot to another manager, immediately.
Factory Once.	initialize (SlotInit)	Set governance and terms, check the module's settings, copy its scopes and fee, call onInstall.
Slot → module ISlotModule. Only callbacks named in the slot's scopes are called.	validateSettings (bytes settings) VIEW	Revert if the settings are invalid. At proposal and install.
	scopes (bytes settings) VIEW	The callbacks it asks for. May depend on the settings. At proposal, install and acceptScopes.
	fee (bytes settings) VIEW	Its share of the tax and who receives it. At proposal, install and acceptFee.
	beforeBuy · beforeSelfAssess (ctx) VIEW	Veto by reverting. Uncapped gas.
	afterBuy · afterRelease · afterLiquidate · afterSettle (ctx)	Told after the fact. 500k gas, always in full (ModuleGasTooLow otherwise); a revert is ignored unless afterCallbacksMustSucceed.
	onInstall (ctx)	Attached. At creation a revert is ignored unless afterCallbacksMustSucceed; attached by queued terms, it is always capped and ignored.
	onUninstall (ctx)	Being replaced or removed. Always capped, a revert always ignored.
	metadata () OPTIONAL	JSON for UIs (IModuleMetadata). The slot never calls it.

Slot methods • reads

All view. Grouped by what they read.
A whole slot, or many, in one call: SlotLens, next page.

READS	METHOD	RETURNS	DOES
Terms	<code>terms()</code>	Terms	Tax terms and module terms in force, together.
	<code>taxTerms()</code>	TaxTerms	Recipient, rate and runway in force.
	<code>moduleTerms()</code>	ModuleTerms	Installed module and its settings.
	<code>recipient()</code>	address	Who receives collected tax, minus the module fee.
	<code>taxRateBps()</code>	uint256	Tax per 30 days, in basis points of the price.
	<code>minRunwaySeconds()</code>	uint256	How long a buyer’s deposit must last at their price.
Module	<code>module()</code>	address	Installed module. Zero = none.
	<code>scopes()</code>	Scopes	The slot’s copy of its scopes, unpacked to booleans.
	<code>fee()</code>	ModuleFee	The slot’s copy of its fee: bps and recipient.
Queue	<code>pending()</code>	Pending	Everything queued for the next buy, as stored.
	<code>hasRipeTerms()</code>	bool	Whether anything queued lands at the next buy. Tax terms and the module ripen on separate clocks.
Governance	<code>currency()</code>	IERC20	Payment token. Zero = native ETH.
	<code>manager()</code>	address	Who proposes terms and accepts module updates.
	<code>mutableTax()</code>	bool	Tax rate and runway may change.
	<code>mutableRecipient()</code>	bool	Recipient may change; a module fee may rise or move.
	<code>mutableModule()</code>	bool	Module may be replaced; new scopes may land.
Occupancy	<code>occupant()</code>	address	Current holder. Zero = vacant.
	<code>price()</code>	uint256	Self-assessed price.
	<code>deposit()</code>	uint256	Escrow that tax is drawn from.
	<code>occupiedSince()</code>	uint64	When the current tenure began.
	<code>tenureId()</code>	uint64	Counts tenures; operator approvals are scoped to one.
	<code>lastSettled()</code>	uint64	Tax is accounted up to this timestamp.
Health	<code>isVacant()</code>	bool	Nobody is seated.
	<code>isInsolvent()</code>	bool	The deposit can no longer cover what is owed.
	<code>taxOwed()</code>	uint256	Tax accrued since the last settle.
	<code>secondsUntillLiquidation()</code>	uint256	Until the deposit runs out. Max if never.
	<code>isOperator(operator)</code>	bool	May act for the current occupant.
Ledger	<code>collectedTax()</code>	uint256	Tax taken, not yet paid out.
	<code>claimableOf(account)</code>	uint256	Payouts waiting for claim.
	<code>debtOf(account)</code>	uint256	Tax the occupant owes beyond an emptied deposit. Zero for anyone not seated.
Quotes	<code>quoteBuy(account, depositAmount)</code>	uint256	Exactly what buy charges: price + deposit. account is ignored.
	<code>minDepositForBuy(price)</code>	uint256	Smallest deposit buy accepts, under ripe queued terms.
	<code>minDepositToHold(price)</code>	uint256	Deposit floor under the terms in force.
Code	<code>version()</code>	uint64	Implementation version.

CONSTANTS • A PUBLIC GETTER EACH

<code>MAX_PRICE</code>	2^{128-1}	<code>MAX_TAX_BPS</code>	10,000	<code>MAX_MIN_RUNWAY</code>	365 days	<code>TERMS_DELAY</code>	1 hour	<code>BASIS_POINTS</code>	10,000	<code>MONTH</code>	30 days
<code>MODULE_CALLBACK_GAS_LIMIT</code>	500,000	<code>NATIVE_PAYOUT_GAS_LIMIT</code>	30,000	<code>TERM_* bits</code>				<code>TAX_RATE</code> 1 · <code>RECIPIENT</code> 2 · <code>MIN_RUNWAY</code> 4 · <code>MODULE</code> 8 · <code>SCOPES</code> 16			

Factory & lens

Both are UUPS proxies (ERC1967Proxy) deployed by DeployProtocol. admin upgrades each. Plain sequential storage, not ERC-7201. Admin errors: NotManager; a zero address: InvalidRecipient.

SlotFactory Creates slots behind one beacon. Upgrading the beacon upgrades every slot.

GROUP	MEMBER	TYPE · RETURNS	DOES
Storage All public.	beacon	UpgradeableBeacon	The beacon every slot delegates to. Owned by the factory, not the admin.
	admin	address	Upgrades the beacon and the factory.
	isSlot (address)	bool	Slots this factory created.
	slotCount	uint256	Slots created so far. With the chain id, the CREATE2 salt of the next one.
Anyone	createSlot (SlotInit init)	address	Deploy a BeaconProxy at a CREATE2 address, initialize it, record it. Emits SlotCreated.
	collectAll (address[] slots)	uint256[]	Pay out collected tax from many slots. Each one is isolated and gets at most COLLECT_GAS (1.5M): a failure reads 0.
	collectFrom (slot)	uint256	Pay out one slot and return what moved. Reverts NotASlot for a slot it did not create.
Admin	upgradeBeacon (implementation)		Point every slot at new code. Emits BeaconUpgraded.
	upgradeToAndCall (implementation, data)		Upgrade the factory itself.
	transferAdmin (next)		Hand over the admin role, beacon power included. Emits AdminTransferred.
Reads	implementation ()	address	The slot code the beacon points at now.
	version ()	uint64	The factory's own code version.
	initializedVersion ()	uint64	Which migration has run in the proxy's storage.
Events	SlotCreated (slot, recipient, creator, currency, module)		The first three indexed. creator is msg.sender.
	BeaconUpgraded (implementation)		Every slot now runs this code.
	AdminTransferred (from, to)		Also emitted at initialize, from zero.

SlotLens Reads a slot through its own getters. Stateless apart from its admin; every answer comes from one block.

GROUP	MEMBER	TYPE · RETURNS	DOES
Storage	admin	address	Upgrades the lens. Holds no other power.
Reads Anyone. None writes.	getSlotInfo (slot)	SlotInfo	Every read on the slot's reads page, in one call. Reverts if slot is not a slot.
	getSlotInfos (address[] slots)	SlotInfo[]	Many slots in one call. Reverts if any one is not a slot.
	getSlotConstants (slot)	SlotConstantsInfo	Every constant the slot runs under, read from the slot.
	moduleUpdate (slot)	ModuleUpdate	What the module declares beside the slot's copy, and whether acceptFee / acceptScopes would change it. Never reverts on a bad module.
	version · initializedVersion ()	uint64	Code version · migration run in the proxy's storage.
Admin	upgradeToAndCall (implementation, data)		Upgrade the lens.
	transferAdmin (next)		Hand over the admin role. Emits AdminTransferred(from, to).

MODULEUPDATE		
currentScopes	uint16	The slot's copy, as scope bits.
currentFee	ModuleFee	The slot's copy.
answered	bool	False; no module, a revert, or an answer out of range.
declaredScopes · declaredFee	uint16 · ModuleFee	What the module says now.
feeDiffers	bool	acceptFee would change the fee now.
scopesDiffer	bool	acceptScopes would queue new scopes.

SLOTCONSTANTSINFO · SLOTLINFO		
maxPrice · maxTaxBps	uint256	MAX_PRICE · MAX_TAX_BPS
basisPoints · month	uint256	BASIS_POINTS · MONTH
moduleCallbackGasLimit	uint256	MODULE_CALLBACK_GAS_LIMIT; nativePayoutGasLimit likewise.
termsDelay · maxMinRunway	uint64 · uint256	TERMS_DELAY · MAX_MIN_RUNWAY
termTaxRate ... termScopes	uint16	The five TERM_* bits.
SlotInfo	struct	One field per slot read: governance, terms, scopes, fee, occupancy, money, pending.